

Understanding Explicit Wait in Selenium

Explicit Wait is a type of wait in Selenium WebDriver used to wait for a certain condition or for a specific element to become available before proceeding further in the code. Unlike **Implicit Wait**, which waits for all elements for a specified time, Explicit Wait applies only to particular elements or conditions and waits until a specific condition is met or the maximum defined wait time is exceeded.

In any UI automation, Explicit Wait is crucial for handling dynamic web elements. Often, certain elements may take time to appear, load, or become clickable (e.g., a dynamic shipping label), and Explicit Wait helps ensure that these elements are interacted with once they are ready.

How Explicit Wait Works

- It is based on the `WebDriverWait` class in Selenium.
- It continuously polls the DOM until the condition is satisfied or the maximum wait time is reached.
- You can define specific conditions using the `ExpectedConditions` class (e.g., visibility, element clickable, presence of an element).

Key Advantages

- **More control:** It applies to specific elements and waits for them under specific conditions.
- **Avoids false failures:** Reduces the chances of a `NoSuchElementException` or `ElementNotInteractableException` by waiting for elements to become visible or clickable.
- **Efficient:** The polling mechanism constantly checks the condition every 500ms (by default) and proceeds once the condition is satisfied, reducing unnecessary waiting.

Methods in WebDriverWait with Real-Time Programming Examples

1. `until(ExpectedConditions<T> isTrue)`

- **Return Type:** T (the type passed as a parameter in the condition)
- **Example:** Wait until the element is clickable.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;
```

```

public class FedExAutomation {

    public static void main(String[] args) {

        System.setProperty("webdriver.chrome.driver", "path-to-chromedriver");

        WebDriver driver = new ChromeDriver();

        driver.get("https://www.fedex.com/en-us/home.html");


        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));


        // Wait until the 'Track' button is clickable

        WebElement trackButton =
wait.until(ExpectedConditions.elementToBeClickable(By.id("trackButtonId")));

        trackButton.click();


        driver.quit();
    }
}

```

2. **until(ExpectedConditions<Boolean> isTrue)**

- **Return Type:** Boolean
- **Example:** Wait until a particular title appears on the page.

```

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

Boolean isTitlePresent = wait.until(ExpectedConditions.titleContains("FedEx Tracking"));

System.out.println("Is title present? " + isTitlePresent);

```

3. **until(ExpectedConditions<WebElement> isTrue)**

- **Return Type:** WebElement
- **Example:** Wait until a FedEx delivery status message is visible.

```

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));

WebElement statusMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("deliveryStatusId")));

```

```
System.out.println("Delivery Status: " + statusMessage.getText());
```

4. **pollingEvery(Duration interval)**

- **Return Type:** WebDriverWait
- **Example:** Set polling to check for the visibility of an element every 1 second.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
wait.pollingEvery(Duration.ofSeconds(1))
```

```
.until(ExpectedConditions.visibilityOfElementLocated(By.id("fedexElement")));
```

5. **ignoring(Class<? extends Throwable> exceptionType)**

- **Return Type:** WebDriverWait
- **Example:** Ignore NoSuchElementException and StaleElementReferenceException during waiting.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
```

```
wait.ignoring(NoSuchElementException.class)
```

```
.ignoring(StaleElementReferenceException.class)
```

```
.until(ExpectedConditions.visibilityOfElementLocated(By.id("dynamicFedExElement")));
```

6. **until(ExpectedConditions<List<WebElement>> isTrue)**

- **Return Type:** List<WebElement>
- **Example:** Wait until a list of elements is present on the page, such as tracking history entries.

```
List<WebElement> trackingHistory =
```

```
wait.until(ExpectedConditions.visibilityOfAllElementsLocatedBy(By.className("trackingHistoryClass")));
```

```
for (WebElement history : trackingHistory) {
```

```
    System.out.println(history.getText());
```

```
}
```

7. **withTimeout(Duration timeout)**

- **Return Type:** WebDriverWait
- **Example:** Set a custom timeout for a specific condition.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));
```

```
wait.withTimeout(Duration.ofSeconds(30))
```

```
.until(ExpectedConditions.visibilityOfElementLocated(By.id("fedexSpecificElement")));
```

8. **ignoring(Exception... exceptions)**

- **Return Type:** WebDriverWait
- **Example:** Ignore multiple exceptions (e.g., NoSuchElementException, StaleElementReferenceException).

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.ignoring(NosuchElementException.class, StaleElementReferenceException.class)  
.until(ExpectedConditions.elementToBeClickable(By.id("fedexClickableElement")));
```

9. **withMessage(String message)**

- **Return Type:** WebDriverWait
- **Example:** Add a custom message in case of failure after wait timeout.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
WebElement element = wait.withMessage("Element not found within time frame")  
.until(ExpectedConditions.presenceOfElementLocated(By.id("trackingNumberField")));
```

10. **until(Function<? super WebDriver, V> isTrue)**

- **Return Type:** V
- **Example:** Create a custom condition using a function (e.g., waiting until a JavaScript variable equals a certain value).

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
Boolean isConditionMet = wait.until(driver -> ((JavascriptExecutor) driver)  
.executeScript("return document.readyState").equals("complete"));  
System.out.println("Is page loaded? " + isConditionMet);
```

Conclusion

Explicit Wait in Selenium gives precise control over waiting for specific elements or conditions, making it ideal for applications like FedEx Ecommerce, where dynamic content loading and element interactions are common. Proper use of WebDriverWait and its methods ensures that your tests run smoothly, avoiding unnecessary failures due to timing issues.

ExpectedConditions Class in Selenium

The ExpectedConditions class in Selenium provides a collection of predefined conditions to use with **Explicit Wait** in WebDriver. It contains several static methods that are used to wait for specific conditions such as the visibility of an element, presence of an element, or a page title, among others.

1. elementToBeClickable(By locator)

- **Return Type:** WebElement
- **Description:** Waits for an element to become clickable.
- **Example:** Waiting for the "Track" button to become clickable.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

WebElement trackButton =
wait.until(ExpectedConditions.elementToBeClickable(By.id("trackButtonId")));

trackButton.click();
```

2. visibilityOfElementLocated(By locator)

- **Return Type:** WebElement
- **Description:** Waits until the specified element is visible.
- **Example:** Waiting for a shipping status label to become visible.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

WebElement statusLabel =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("shippingStatusId")));

System.out.println(statusLabel.getText());
```

3. titleContains(String title)

- **Return Type:** Boolean
- **Description:** Waits until the title of the page contains the specified string.
- **Example:** Waiting for the page title to contain "FedEx Tracking".

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

Boolean isTitleCorrect = wait.until(ExpectedConditions.titleContains("FedEx Tracking"));

System.out.println("Is title correct: " + isTitleCorrect);
```

4. presenceOfElementLocated(By locator)

- **Return Type:** WebElement
- **Description:** Waits until the specified element is present in the DOM.
- **Example:** Waiting for the presence of the tracking number input field.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
WebElement trackingInput =  
wait.until(ExpectedConditions.presenceOfElementLocated(By.id("trackingInputId")));  
trackingInput.sendKeys("123456789");
```

5. alertIsPresent()

- **Return Type:** Alert
- **Description:** Waits for an alert to be present.
- **Example:** Handling an alert that appears after entering an invalid tracking number.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Alert alert = wait.until(ExpectedConditions.alertIsPresent());  
alert.accept();
```

6. textToBePresentInElement(By locator, String text)

- **Return Type:** Boolean
- **Description:** Waits until the specified text is present in the given element.
- **Example:** Waiting for the "Delivered" text to appear in the delivery status label.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));
```

```
Boolean isDelivered =  
wait.until(ExpectedConditions.textToBePresentInElement(By.id("deliveryStatusId"), "Delivered"));  
System.out.println("Package delivered: " + isDelivered);
```

7. invisibilityOfElementLocated(By locator)

- **Return Type:** Boolean
- **Description:** Waits until the specified element is no longer visible.
- **Example:** Waiting for a loading spinner to disappear after tracking a shipment.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Boolean isSpinnerGone =  
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("loadingSpinnerId")));  
System.out.println("Loading spinner gone: " + isSpinnerGone);
```

8. elementToBeSelected(By locator)

- **Return Type:** Boolean
- **Description:** Waits until the specified element is selected (for radio buttons, checkboxes, etc.).
- **Example:** Waiting for the shipping method radio button to be selected.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Boolean isShippingSelected =  
wait.until(ExpectedConditions.elementToBeSelected(By.id("shippingMethodRadioButton")));
```

```
System.out.println("Shipping method selected: " + isShippingSelected);
```

9. elementSelectionStateToBe(By locator, boolean selected)

- **Return Type:** Boolean
- **Description:** Waits until the element's selection state is either selected or deselected, as specified.
- **Example:** Waiting for a checkbox to be selected.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Boolean isCheckboxSelected =  
wait.until(ExpectedConditions.elementSelectionStateToBe(By.id("termsCheckbox"), true));
```

```
System.out.println("Checkbox selected: " + isCheckboxSelected);
```

10. visibilityOf(WebElement element)

- **Return Type:** WebElement
- **Description:** Waits until the specified WebElement is visible.
- **Example:** Waiting for the "Track" button to become visible.

```
WebElement trackButton = driver.findElement(By.id("trackButtonId"));
```

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
wait.until(ExpectedConditions.visibilityOf(trackButton));
```

```
trackButton.click();
```

11. frameToBeAvailableAndSwitchToIt(By locator)

- **Return Type:** WebDriver
- **Description:** Waits until the frame is available and switches to it.
- **Example:** Waiting for a payment gateway iframe to load and switching to it.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("paymentFrameId")));
```

12. numberOfElementsToBe(By locator, int number)

- **Return Type:** Boolean
- **Description:** Waits until the number of elements located by the locator equals the specified number.
- **Example:** Waiting until the correct number of tracking results are displayed.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
```

```
Boolean areResultsLoaded =
```

```
wait.until(ExpectedConditions.numberOfElementsToBe(By.className("trackingResult"), 10));
```

```
System.out.println("Are 10 results loaded: " + areResultsLoaded);
```

13. stalenessOf(WebElement element)

- **Return Type:** Boolean
- **Description:** Waits until the given element is no longer attached to the DOM (useful for stale element issues).
- **Example:** Waiting for an outdated element to be removed from the DOM.

```
WebElement oldElement = driver.findElement(By.id("oldElementId"));
```

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Boolean isElementStale = wait.until(ExpectedConditions.stalenessOf(oldElement));
```

```
System.out.println("Is element stale: " + isElementStale);
```

14. urlToBe(String url)

- **Return Type:** Boolean
- **Description:** Waits until the current URL matches the specified URL.
- **Example:** Waiting for the URL to match after navigating to the order confirmation page.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
Boolean isUrlCorrect = wait.until(ExpectedConditions.urlToBe("https://www.fedex.com/en-us/track/confirmation.html"));
```

```
System.out.println("Is URL correct: " + isUrlCorrect);
```

15. elementIfVisible(WebElement element)

- **Return Type:** WebElement
- **Description:** Returns the element if visible; otherwise returns null.

- **Example:** Waiting for an element to be visible and return it.

```
WebElement element = driver.findElement(By.id("someId"));
```

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
WebElement visibleElement = wait.until(ExpectedConditions.elementIfVisible(element));
```

Conclusion

The ExpectedConditions class in Selenium is an essential tool for waiting on dynamic conditions in the DOM. In the context of the **FedEx Ecommerce** project, using these conditions allows for smoother interactions with dynamic elements such as tracking status, loading indicators, or complex forms. These methods ensure the automation tests handle delays, dynamic content, or any unexpected behavior in web applications.

By Mahasweta S.